

Human-in-the-Loop Adaptive Image Tagging System with Continuous Learning

Deepanshu Jain

School of Computing and Augmented Intelligence

Arizona State University

Tempe, AZ, USA

djain63@asu.edu

Abstract—This paper presents a human-in-the-loop adaptive image tagging system that combines pretrained vision models, user feedback, and continuous retraining to improve tag quality over time. The system addresses the limitation of static pretrained taggers, which provide strong initial predictions but cannot easily adapt to user-specific preferences or domain requirements. To solve this, the proposed architecture integrates RAM++ for base tag generation, CLIP for semantic image embeddings, and a lightweight adaptive multi-label classifier trained on feedback collected through a web-based interface. Users can review predicted tags, mark them as correct, partially correct, or incorrect, and add missing tags, allowing the system to build a growing supervision dataset from real interactions. A backend retraining pipeline converts this feedback into training targets and periodically updates the adaptive model, which is then reloaded into the inference pipeline. The system also includes a score-aware tag merging strategy, database-backed traceability, and cloud deployment support. This project demonstrates a practical approach for building continuously improving image tagging systems without requiring large manually labeled datasets at the start.

Index Terms—human-in-the-loop learning, image tagging, adaptive classification, CLIP, RAM++, multi-label learning, feedback-driven learning, continuous retraining

I. INTRODUCTION

Image tagging is a fundamental task in computer vision, with applications in search, recommendation systems, content moderation, and digital asset management. Recent advances in pretrained vision-language models have significantly improved the ability to generate descriptive tags for images without requiring large labeled datasets. However, such pretrained systems are inherently static and often fail to align with user-specific requirements or domain-specific tagging needs.

Traditional approaches to improving tagging performance involve training custom models on large annotated datasets. While effective, this process is expensive, time-consuming, and difficult to maintain as requirements evolve. In many real-world applications, user preferences and semantic expectations vary over time, making it necessary for tagging systems to adapt dynamically rather than remain fixed after deployment.

Human-in-the-loop learning offers a practical solution to this problem by incorporating user feedback directly into the learning process. Instead of relying solely on pre-collected datasets, the system continuously gathers supervision signals from user interactions and uses them to refine its predictions.

This enables gradual alignment with user expectations while maintaining the benefits of pretrained models.

In this work, we present a human-in-the-loop adaptive image tagging system that combines pretrained models, semantic feature extraction, and a lightweight trainable classifier with an automated retraining pipeline. The system uses a pretrained RAM++ model to generate initial tags and a CLIP-based encoder to extract semantic image embeddings. A custom multi-label classifier is trained on feedback collected through a web interface, where users can validate predicted tags, mark incorrect ones, and provide additional tags. The system periodically converts this feedback into training data and updates the adaptive model, enabling continuous improvement over time.

The main contributions of this project are as follows. First, we design a modular full-stack architecture that integrates frontend interaction, backend services, machine learning inference, and automated retraining into a unified system. Second, we introduce a feedback-driven training strategy that incorporates positive, negative, and partial supervision signals for multi-label classification. Third, we develop a continuously learning pipeline that updates the model based on real user interactions without requiring manual dataset curation. Finally, we implement a score-aware tag merging strategy that combines pretrained and learned predictions to improve final tagging quality.

II. SYSTEM OVERVIEW

The proposed system is a human-in-the-loop adaptive image tagging framework that combines pretrained models, semantic feature extraction, and continuous learning through user feedback. The goal is to provide high-quality image tags while enabling the system to improve over time based on real user interactions.

The overall workflow of the system follows a closed feedback loop. Given an input image, a pretrained tagging model generates initial tags, which are then refined using an adaptive classifier. The combined predictions are presented to the user through an interactive interface, where the user can validate, modify, or extend the generated tags. This feedback is stored and later used to retrain the adaptive model, enabling the system to progressively align with user expectations.

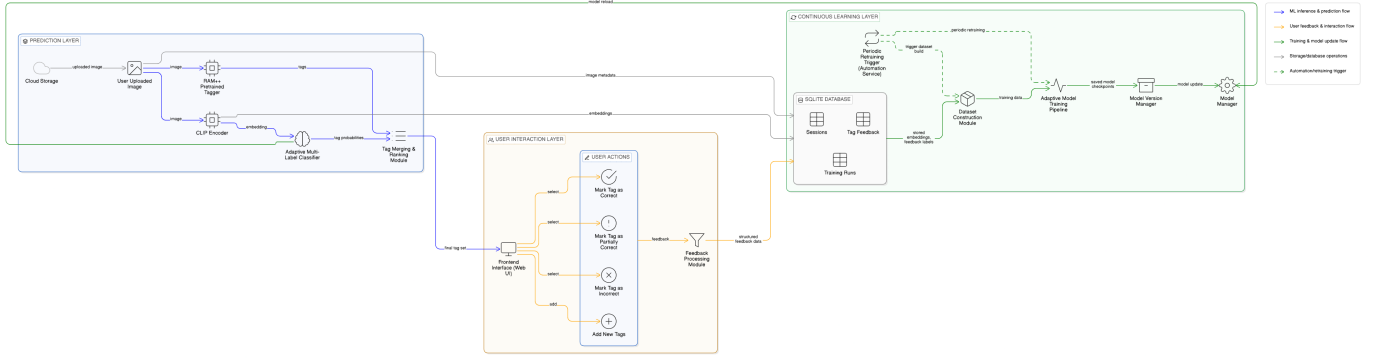


Fig. 1. End-to-end workflow of the human-in-the-loop adaptive image tagging system.

At a high level, the system consists of three primary stages: prediction, feedback collection, and continuous learning. During the prediction stage, the input image is processed by a pretrained RAM++ model to generate base tags, while a CLIP encoder produces a semantic embedding of the image. This embedding is passed to an adaptive multi-label classifier that predicts tag probabilities based on previously learned patterns. The outputs of both models are merged to produce the final set of tags.

In the feedback stage, users interact with the predicted tags by marking them as correct, partially correct, or incorrect, and by adding missing tags. This interaction serves as a direct supervision signal and is stored in a structured database along with the corresponding image embeddings.

In the continuous learning stage, the system periodically constructs a training dataset from accumulated feedback and retrain the adaptive classifier. The updated model is then reloaded into the inference pipeline, allowing subsequent predictions to reflect the learned improvements.

This iterative process enables the system to evolve from a generic pretrained model into a personalized tagging system without requiring a large manually curated dataset at initialization.

III. SYSTEM ARCHITECTURE

The proposed system follows a modular multi-layered architecture that separates user interaction, application logic, machine learning inference, and data management. This design improves scalability, maintainability, and enables independent evolution of system components. The architecture is divided into four primary layers: frontend interface, API layer, service layer, and data layer.

A. Frontend Interface

The frontend serves as the primary interaction point between the user and the system. It allows users to upload images, visualize predicted tags, and provide feedback. The interface presents predicted tags and enables users to mark each tag as correct, partially correct, or incorrect. Additionally, users can introduce new tags that were not generated by the system.

This layer plays a critical role beyond visualization, as it acts as a structured data collection mechanism. All user interactions are captured and forwarded to the backend as supervision signals, forming the foundation for continuous learning.

B. API Layer

The API layer is implemented using FastAPI and serves as the communication bridge between the frontend and backend services. It exposes endpoints for prediction, feedback submission, and administrative control of the training pipeline.

The primary endpoints include image-based tag prediction and feedback submission. The prediction endpoint processes uploaded images and returns generated tags, while the feedback endpoint accepts structured user annotations and stores them for future training. The API layer ensures input validation, request handling, and seamless integration between user-facing and backend components.

C. Service Layer

The service layer contains the core application logic and orchestrates interactions between machine learning models, database operations, and external triggers. It is composed of modular services responsible for different system functionalities.

The inference service manages the prediction pipeline, including preprocessing, model inference, and tag merging. The feedback service processes user annotations and converts them into structured records suitable for storage and training. The training job service handles model retraining, including dataset preparation and execution of training routines. Additionally, an automation service periodically monitors the system state and triggers retraining when sufficient new data is available.

This separation of concerns allows each service to be independently maintained and extended without affecting the overall system behavior.

D. Data Layer

The data layer is responsible for persistent storage of all system information, including user sessions, feedback, and training metadata. A lightweight relational database is used

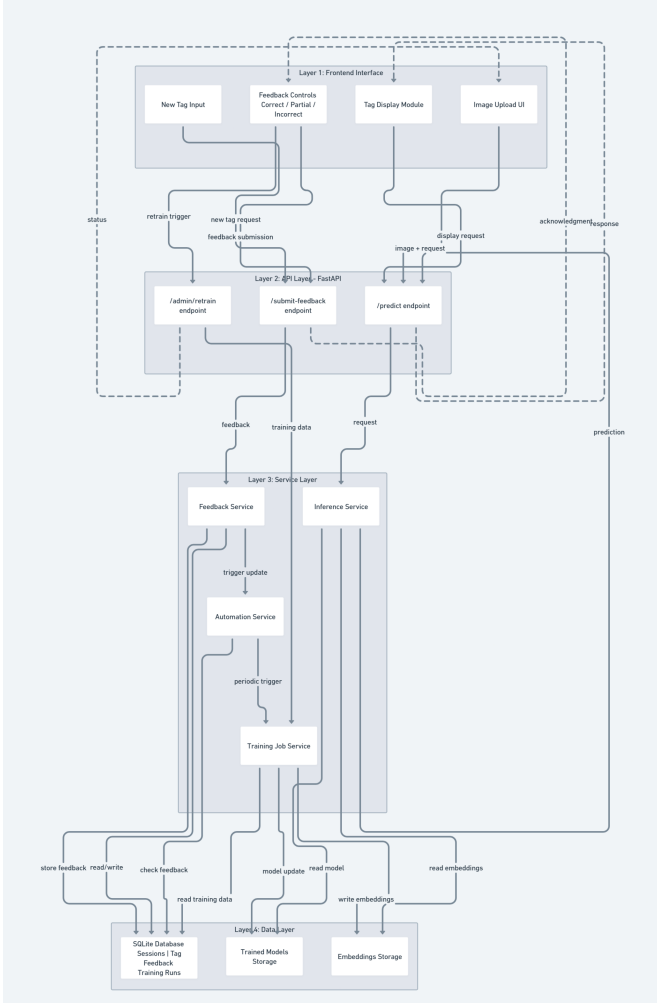


Fig. 2. Layered architecture of the system showing frontend, API, service, and data layers.

to store structured records such as image sessions, tag-level feedback, and training history.

The database schema is designed to support traceability, enabling the system to associate training runs with the exact feedback data used to generate them. This allows for reproducibility, debugging, and analysis of model improvements over time.

In addition to structured data, the system stores image embeddings and trained model artifacts in dedicated directories. This hybrid storage approach ensures efficient access to both relational data and large numerical representations required for machine learning tasks.

IV. MACHINE LEARNING DESIGN

The machine learning component of the system is designed to combine the strengths of pretrained models with the adaptability of a lightweight trainable classifier. Instead of training a model from scratch, the system leverages pretrained models for initial predictions and uses user feedback to incrementally improve performance through a secondary adaptive model.

A. Base Tag Generation using RAM++

The system uses a pretrained RAM++ model as the primary source of initial tags. This model provides strong zero-shot tagging capability and addresses the cold-start problem by generating meaningful tags without requiring prior training on application-specific data.

However, since the RAM++ model is static and cannot adapt to evolving user requirements, its outputs are treated as baseline predictions rather than final results.

B. Semantic Feature Extraction using CLIP

To enable efficient learning, the system uses a CLIP-based encoder to convert input images into fixed-length embedding vectors. These embeddings capture high-level semantic information and serve as the input features for the adaptive classifier.

Using embeddings instead of raw images significantly reduces the computational cost of training and allows the system to reuse pretrained representations for downstream learning tasks.

C. Adaptive Multi-Label Classifier

The adaptive model is implemented as a lightweight neural network that maps image embeddings to tag probabilities. The model follows a multi-label classification setup, where each tag is predicted independently.

The architecture consists of a fully connected feedforward network with a hidden layer, ReLU activation, and dropout regularization. The final layer outputs a probability score for each tag in the vocabulary using a sigmoid activation function.

This design enables the model to learn personalized tagging patterns from user feedback while remaining computationally efficient for frequent retraining.

D. Training Strategy with Feedback Signals

The training process is driven entirely by user feedback collected during system interaction. Each prediction session produces supervision signals based on how users evaluate the generated tags.

The system incorporates three types of feedback signals: correct labels, incorrect labels, and partially correct labels. Correct labels and user-added tags are treated as positive supervision, while incorrect labels provide explicit negative supervision. Partially correct labels are treated as soft supervision signals, indicating that the prediction is directionally relevant but incomplete.

To incorporate these signals, the system constructs training targets using a combination of binary labels and per-label weights. Positive labels are assigned a target value of 1, negative labels are assigned a target value of 0, and partial labels are assigned an intermediate value with reduced weight.

The model is trained using a weighted binary cross-entropy loss, where each label contributes to the loss based on its assigned weight. This allows the system to learn from both positive and negative feedback while handling uncertainty in partial labels.

E. Tag Merging and Prediction Refinement

The final output of the system is obtained by combining predictions from the pretrained model and the adaptive classifier. Instead of simple concatenation, the system employs a score-based merging strategy that ranks tags based on confidence scores and prioritizes tags supported by both models.

This approach improves precision while maintaining diversity in predictions, ensuring that the final tag set reflects both general knowledge and user-specific adaptations.

V. MACHINE LEARNING DESIGN

The machine learning component of the system is designed to combine the strengths of pretrained models with the adaptability of a lightweight trainable classifier. Instead of training a model from scratch, the system leverages pretrained models for initial predictions and uses user feedback to incrementally improve performance through a secondary adaptive model.

A. Base Tag Generation using RAM++

The system uses a pretrained RAM++ model as the primary source of initial tags. This model provides strong zero-shot tagging capability and addresses the cold-start problem by generating meaningful tags without requiring prior training on application-specific data.

However, since the RAM++ model is static and cannot adapt to evolving user requirements, its outputs are treated as baseline predictions rather than final results.

B. Semantic Feature Extraction using CLIP

To enable efficient learning, the system uses a CLIP-based encoder to convert input images into fixed-length embedding vectors. These embeddings capture high-level semantic information and serve as the input features for the adaptive classifier.

Using embeddings instead of raw images significantly reduces the computational cost of training and allows the system to reuse pretrained representations for downstream learning tasks.

C. Adaptive Multi-Label Classifier

The adaptive model is implemented as a lightweight neural network that maps image embeddings to tag probabilities. The model follows a multi-label classification setup, where each tag is predicted independently.

The architecture consists of a fully connected feedforward network with a hidden layer, ReLU activation, and dropout regularization. The final layer outputs a probability score for each tag in the vocabulary using a sigmoid activation function.

This design enables the model to learn personalized tagging patterns from user feedback while remaining computationally efficient for frequent retraining.

D. Training Strategy with Feedback Signals

The training process is driven entirely by user feedback collected during system interaction. Each prediction session produces supervision signals based on how users evaluate the generated tags.

The system incorporates three types of feedback signals: correct labels, incorrect labels, and partially correct labels. Correct labels and user-added tags are treated as positive supervision, while incorrect labels provide explicit negative supervision. Partially correct labels are treated as soft supervision signals, indicating that the prediction is directionally relevant but incomplete.

To incorporate these signals, the system constructs training targets using a combination of binary labels and per-label weights. Positive labels are assigned a target value of 1, negative labels are assigned a target value of 0, and partial labels are assigned an intermediate value with reduced weight.

The model is trained using a weighted binary cross-entropy loss, where each label contributes to the loss based on its assigned weight. This allows the system to learn from both positive and negative feedback while handling uncertainty in partial labels.

E. Tag Merging and Prediction Refinement

The final output of the system is obtained by combining predictions from the pretrained model and the adaptive classifier. Instead of simple concatenation, the system employs a score-based merging strategy that ranks tags based on confidence scores and prioritizes tags supported by both models.

This approach improves precision while maintaining diversity in predictions, ensuring that the final tag set reflects both general knowledge and user-specific adaptations.

VI. CONTINUOUS LEARNING PIPELINE

A key feature of the proposed system is its ability to improve over time through a continuous learning pipeline driven by user interaction. Rather than treating prediction and training as separate offline processes, the system integrates them into a closed-loop workflow in which user feedback directly influences future model behavior.

A. Prediction and Feedback Collection

Each learning cycle begins when a user uploads an image and receives a set of predicted tags. These tags are generated through the combined output of the pretrained RAM++ model and the adaptive classifier. The predictions are then presented through the frontend interface, where the user can review each tag and provide structured feedback.

The feedback mechanism allows users to classify generated tags as correct, partially correct, or incorrect, while also adding missing tags that were not suggested by the system. This design enables the collection of both positive and negative supervision signals from ordinary usage, transforming the interface into a practical annotation mechanism.

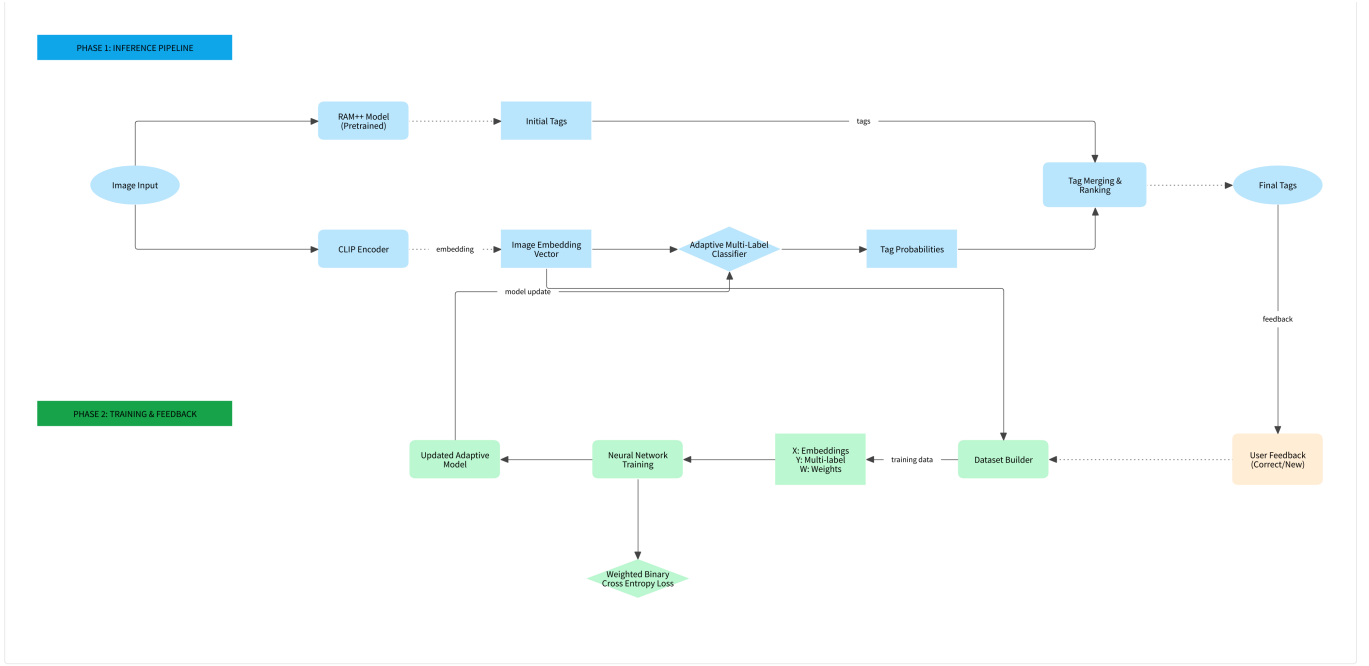


Fig. 3. Machine learning pipeline showing feature extraction, adaptive classification, and feedback-driven training.

B. Storage and Traceability

Once feedback is submitted, the system stores the corresponding session data, reviewed tags, and image embedding in persistent storage. This information is recorded in a relational database and linked through session identifiers, allowing the full history of a prediction event to be tracked.

In addition to user interaction records, the system stores metadata about training runs and their associated input sessions. This traceability makes it possible to identify which feedback samples contributed to a model update and supports reproducibility during debugging or later system analysis.

C. Dataset Construction

The retraining process begins by constructing a supervised dataset from accumulated feedback. For each eligible session, the stored CLIP embedding is used as the input feature representation, while the user-reviewed tags are transformed into multi-label targets and per-label weights.

This stage converts raw interaction data into a structured training format suitable for the adaptive classifier. By using stored embeddings rather than recomputing visual features from raw images, the system reduces retraining cost and simplifies repeated updates.

D. Automated Retraining

The system includes an automation service that periodically checks whether sufficient new supervision data is available to justify retraining. If the retraining criteria are satisfied, the pipeline invokes the dataset builder, launches the training process, stores the resulting model checkpoint, and updates the corresponding training metadata.

This automation enables the system to adapt without manual retraining intervention. It also ensures that the adaptive model remains synchronized with newly collected user feedback while preserving previous model versions for inspection and comparison.

E. Model Reload and Closed-Loop Adaptation

After a successful training run, the updated adaptive model is reloaded into the inference pipeline. Subsequent predictions therefore reflect patterns learned from prior user feedback, allowing the system to gradually shift from generic pretrained behavior toward more personalized and application-specific tagging.

This closed-loop design is central to the proposed architecture. It allows the system to remain useful from the beginning through pretrained inference, while continuously improving through lightweight retraining as more feedback becomes available.

VII. IMPLEMENTATION DETAILS

The system is implemented as a full-stack application that integrates a web-based user interface, a backend API, machine learning components, and cloud-based deployment. The implementation emphasizes modularity, allowing each component to be developed, tested, and extended independently.

A. Technology Stack

The backend is developed using FastAPI, which provides a lightweight and efficient framework for building RESTful APIs. Machine learning components are implemented using PyTorch, enabling flexible model design and efficient training. The frontend is built using standard web technologies,

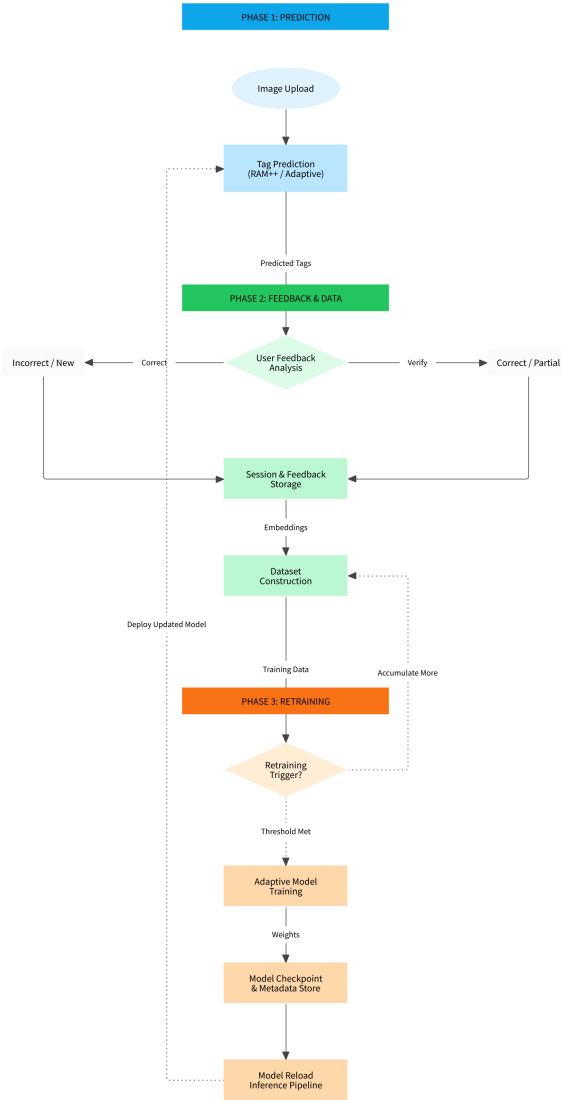


Fig. 4. Continuous learning pipeline showing feedback collection, dataset construction, retraining, and model reload.

including HTML, CSS, and JavaScript, providing a responsive interface for user interaction.

A lightweight SQLite database is used for structured data storage, while NumPy is used for handling embedding data and intermediate training artifacts. This combination allows for rapid development while maintaining sufficient performance for the scale of the application.

B. Backend Design and API Endpoints

The backend exposes a set of endpoints that handle prediction, feedback submission, and administrative operations. The prediction endpoint accepts an uploaded image, processes it through the inference pipeline, and returns generated tags along with a session identifier. The feedback endpoint receives structured user annotations and stores them in the database along with the corresponding embedding references.

Administrative endpoints are used to control retraining operations, including triggering retraining, checking training status, and reloading updated models. This separation between user-facing and administrative functionality ensures clean system organization and easier maintenance.

C. Model and Data Management

The system maintains a clear separation between pretrained models and dynamically trained models. Pretrained models, such as RAM++ and CLIP, are stored in a dedicated directory and remain unchanged during operation. In contrast, adaptive model checkpoints are versioned and stored after each training run, allowing the system to track model evolution over time.

Image embeddings are stored as serialized NumPy arrays, enabling efficient reuse during retraining without recomputing features. Training datasets are generated dynamically and stored in intermediate files, including feature matrices, label vectors, and weight matrices.

D. Automation and Deployment

The system includes an automation service that periodically monitors the availability of new feedback data and triggers retraining when predefined conditions are met. This enables continuous model updates without requiring manual intervention.

The application is deployed on a cloud-based virtual machine using an Amazon EC2 instance. The backend server is hosted using a production-ready ASGI server, allowing external access through configured network ports. This deployment setup ensures that the system can be accessed remotely while supporting ongoing development and updates.

E. System Integration

All components of the system are integrated through well-defined interfaces. The frontend communicates with the backend through HTTP requests, the backend interacts with the database and file storage for persistence, and the machine learning components are encapsulated within the service layer for inference and training.

This modular integration allows the system to maintain clarity in data flow and ensures that changes in one component, such as the training pipeline or frontend interface, do not require significant modifications in other parts of the system.

VIII. IMPLEMENTATION DETAILS

The system is implemented as a full-stack application that integrates a web-based user interface, a backend API, machine learning components, and cloud-based deployment. The implementation emphasizes modularity, allowing each component to be developed, tested, and extended independently.

A. Technology Stack

The backend is developed using FastAPI, which provides a lightweight and efficient framework for building RESTful APIs. Machine learning components are implemented using PyTorch, enabling flexible model design and efficient training. The frontend is built using standard web technologies,

including HTML, CSS, and JavaScript, providing a responsive interface for user interaction.

A lightweight SQLite database is used for structured data storage, while NumPy is used for handling embedding data and intermediate training artifacts. This combination allows for rapid development while maintaining sufficient performance for the scale of the application.

B. Backend Design and API Endpoints

The backend exposes a set of endpoints that handle prediction, feedback submission, and administrative operations. The prediction endpoint accepts an uploaded image, processes it through the inference pipeline, and returns generated tags along with a session identifier. The feedback endpoint receives structured user annotations and stores them in the database along with the corresponding embedding references.

Administrative endpoints are used to control retraining operations, including triggering retraining, checking training status, and reloading updated models. This separation between user-facing and administrative functionality ensures clean system organization and easier maintenance.

C. Model and Data Management

The system maintains a clear separation between pretrained models and dynamically trained models. Pretrained models, such as RAM++ and CLIP, are stored in a dedicated directory and remain unchanged during operation. In contrast, adaptive model checkpoints are versioned and stored after each training run, allowing the system to track model evolution over time.

Image embeddings are stored as serialized NumPy arrays, enabling efficient reuse during retraining without recomputing features. Training datasets are generated dynamically and stored in intermediate files, including feature matrices, label vectors, and weight matrices.

D. Automation and Deployment

The system includes an automation service that periodically monitors the availability of new feedback data and triggers retraining when predefined conditions are met. This enables continuous model updates without requiring manual intervention.

The application is deployed on a cloud-based virtual machine using an Amazon EC2 instance. The backend server is hosted using a production-ready ASGI server, allowing external access through configured network ports. This deployment setup ensures that the system can be accessed remotely while supporting ongoing development and updates.

E. System Integration

All components of the system are integrated through well-defined interfaces. The frontend communicates with the backend through HTTP requests, the backend interacts with the database and file storage for persistence, and the machine learning components are encapsulated within the service layer for inference and training.

This modular integration allows the system to maintain clarity in data flow and ensures that changes in one component,

such as the training pipeline or frontend interface, do not require significant modifications in other parts of the system.

IX. RESULTS AND OBSERVATIONS

TABLE I
QUALITATIVE COMPARISON BEFORE AND AFTER FEEDBACK

Image	Before Feedback	After Feedback
Image 1	generic tags	refined tags
Image 2	noisy tags	filtered tags

Due to the scope of this project and the absence of large-scale deployment, quantitative evaluation on extensive real-world datasets was not performed. Instead, the system was evaluated through functional validation and controlled small-scale experiments to verify the effectiveness of the proposed architecture and learning pipeline.

A. End-to-End System Validation

The system was tested across multiple image inputs to validate the complete workflow, including prediction, feedback collection, dataset construction, retraining, and model reloading. The results confirmed that the system successfully processes user input, generates tags, records structured feedback, and incorporates this feedback into subsequent training cycles.

The integration between frontend interaction, backend services, and machine learning components was observed to function reliably, demonstrating the correctness of the system design and implementation.

B. Qualitative Behavior of Adaptive Learning

Initial predictions generated by the pretrained model were generally broad and sometimes included irrelevant or overly generic tags. After incorporating user feedback and retraining the adaptive model on a small set of annotated samples, the system exhibited improved alignment with user expectations in subsequent predictions.

Specifically, tags that were consistently marked as incorrect became less likely to appear, while frequently confirmed or user-added tags were more likely to be predicted. This behavior indicates that the adaptive model successfully learns from both positive and negative supervision signals, even with limited data.

C. Impact of Feedback Types

The introduction of negative labels and partially correct labels provided more informative supervision compared to earlier versions of the system that relied only on positive signals. Negative feedback contributed to reducing irrelevant predictions, while partial feedback enabled the model to capture nuanced relationships between tags and image content.

Although the dataset size remained small, these additional feedback signals improved the model’s ability to refine predictions beyond simple reinforcement of existing tags.

D. System-Level Observations

The continuous learning pipeline demonstrated stable behavior under repeated retraining cycles. The use of precomputed embeddings reduced computational overhead during dataset construction, allowing retraining to be performed efficiently.

The automated retraining mechanism successfully triggered training runs when sufficient feedback data was available and updated the adaptive model without manual intervention. The system maintained consistent performance across iterations, with no observed degradation due to retraining.

E. Limitations of Evaluation

The primary limitation of the evaluation is the lack of large-scale user data, which restricts the ability to perform statistical analysis or benchmark the system against existing tagging models. As a result, the observations are qualitative and based on limited interaction scenarios.

However, the results demonstrate that the system behaves as intended and that the proposed architecture is capable of supporting scalable, feedback-driven learning in a real-world deployment setting.

X. LIMITATIONS

Despite demonstrating the effectiveness of a human-in-the-loop adaptive image tagging framework, the current system has several limitations that restrict its performance and scalability.

A. Limited Scale of Training Data

The system currently relies on user-generated feedback collected during interaction. In the absence of large-scale deployment, the volume of available training data remains limited. As a result, the adaptive model is trained on relatively small datasets, which constrains its ability to generalize across diverse image domains and complex tagging scenarios.

B. Single-User Feedback Bias

Since the system has primarily been evaluated in a single-user setting, the collected feedback reflects a narrow set of preferences and labeling patterns. This introduces bias into the adaptive model and limits its applicability to broader user populations. In a real-world deployment, diverse user interactions would be necessary to improve robustness and generalization.

C. Simplified Model Architecture

The adaptive classifier is implemented as a lightweight feedforward neural network to enable fast retraining and low computational overhead. While this design is efficient, it limits the model's capacity to capture more complex semantic relationships that could be learned using deeper or more advanced architectures.

D. Absence of Quantitative Evaluation Metrics

Due to the limited dataset size, the system has not been evaluated using standard quantitative metrics such as precision, recall, or F1-score. As a result, performance assessment is based primarily on qualitative observations rather than statistically significant benchmarks.

E. Dependence on Pretrained Models

The overall system performance remains partially dependent on the quality of pretrained models such as RAM++ and CLIP. While these models provide strong initial predictions, any inherent biases or limitations in their training data may propagate into the system's outputs.

F. Lack of Semantic Tag Structure

The current system treats tags as independent labels without incorporating hierarchical or semantic relationships between them. This limits the system's ability to understand contextual connections between tags or to generalize across related concepts.

G. Infrastructure Constraints

The system is deployed on a single cloud instance with limited computational resources. While sufficient for development and testing, this setup may not scale efficiently under high user load or large-scale retraining scenarios without further optimization or distributed infrastructure.

XI. FUTURE WORK

The current system provides a functional foundation for adaptive image tagging; however, several directions can be explored to improve its performance, scalability, and applicability in real-world scenarios.

A. Scalable Data Collection and Multi-User Learning

A natural extension of the system is deployment in a multi-user environment, where feedback from diverse users can be aggregated to improve model generalization. Incorporating large-scale interaction data would enable the adaptive model to learn richer patterns and reduce bias introduced by single-user feedback.

B. Advanced Model Architectures

Future work may explore more expressive model architectures for the adaptive classifier, such as transformer-based or graph-based models. These approaches could better capture complex semantic relationships between image features and tags, potentially improving prediction quality beyond what is achievable with a shallow feedforward network.

C. Improved Handling of Partial Feedback

While the current system incorporates partial labels as soft supervision signals, more sophisticated strategies could be developed. For example, hierarchical tagging or semantic similarity measures could be used to better interpret partial correctness and guide learning in a more structured manner.

D. Ontology-Aware Tag Representation

Introducing a structured representation of tags, such as a taxonomy or ontology, could enhance the system’s ability to understand relationships between concepts. This would allow the model to generalize across related tags and produce more contextually consistent predictions.

E. Quantitative Evaluation Framework

Developing a formal evaluation pipeline using metrics such as precision, recall, and F1-score would enable systematic comparison of model performance across training iterations. This would also facilitate benchmarking against existing image tagging approaches.

F. Enhanced Merging and Ranking Strategies

The current score-based merging strategy can be further improved by incorporating learning-based ranking mechanisms. Future work may explore training a dedicated ranking model or using attention-based methods to better combine predictions from pretrained and adaptive components.

G. Distributed Training and Deployment

To support large-scale usage, the system can be extended to a distributed architecture with scalable storage and parallel training capabilities. This would allow efficient handling of high volumes of user feedback and enable faster model updates in production environments.

XII. CONCLUSION

This paper presented a human-in-the-loop adaptive image tagging system that integrates pretrained models, semantic feature extraction, and continuous learning through user feedback. The proposed architecture combines the strengths of pretrained tagging models with a lightweight adaptive classifier, enabling the system to improve over time without requiring large annotated datasets at initialization.

The system demonstrates a practical approach to bridging the gap between static pretrained models and dynamic real-world requirements. By incorporating structured user feedback into the training process, the system evolves toward better alignment with user expectations while maintaining computational efficiency through the use of precomputed embeddings and automated retraining.

The modular design of the system, including its frontend interface, backend services, and machine learning pipeline, supports extensibility and scalability. Although current evaluation is limited to small-scale usage, the architecture is well-suited for deployment in larger environments where continuous feedback can drive significant performance improvements.

Overall, this work highlights the potential of feedback-driven learning systems and provides a foundation for building adaptive, user-aligned image tagging solutions in practical applications.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [4] F. Chollet, *Deep Learning with Python*. Shelter Island, NY, USA: Manning Publications, 2017.
- [5] PyTorch Documentation, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Available: <https://pytorch.org/docs>
- [6] OpenAI, “CLIP Model Documentation,” Available: <https://github.com/openai/CLIP>
- [7] FastAPI Documentation, “FastAPI Framework,” Available: <https://fastapi.tiangolo.com/>
- [8] Mozilla Developer Network (MDN), “Web Development Documentation,” Available: <https://developer.mozilla.org/>
- [9] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [10] T. O’Reilly, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.